# Damn Fast DFIR — Arkime Cheat Sheet

by MatthySoft

Session search expressions, field reference, hunting queries, UI workflow, API and offline PCAP import.

OFFLINE FIELD REFERENCE
A4 print edition
20 Oct 2026
src: src/arkime_content.html

## SEARCH Expression syntax

| EXPRESSION | MEANING |
|---|---|
| `ip.src == 10.1.1.5` | equals |
| `ip.src != 10.1.1.5` | not equal |
| `A && B  A \|\| B  !(A)` | and / or / not; group with ( ) |
| `port.dst > 1024  (< <= >=)` | numeric comparison |
| `port.dst == [80,443,8080]` | any of the list (OR) |
| `port.dst != [80,443]` | none of the list |
| `tags == ]case42,triaged[` | ALL of the list (multi-value fields) |
| `host.http == *evil*` | wildcard: `*` any run, `?` one char |
| `host.dns == /.*\.(top\|xyz)/` | regex (Lucene); always matches the whole value, no `^ $` |
| `http.uri == "*/a b?x=1*"` | quote values with spaces or specials |
| `http.user-agent == EXISTS!` | field present; `!= EXISTS!` = absent |
| `ip == 10.0.0.0/8` | CIDR on any IP field |
| `ip.dst == 10.1.1.5:443` | IP and port in one term |
| `ip.dst != $internal` | a **Shortcut** list (Settings › Shortcuts) |

**Field names:** start typing in the search bar and autocomplete lists fields and values. The full field list for this build is under the **?** (Help) page › Fields, served by the viewer itself, so it works offline.

## READ FIRST Time range & imported PCAPs

- Arkime filters on **packet time**, not import time. An old capture is invisible under "Last hour": pick **All (careful)** or a custom start/stop that covers the capture. URL form: `&date=-1`.
- **Bounding** picks which session time is tested: *Last Packet* (default), *First Packet*, *Bounded* (both inside), *Session Overlaps*, *Database* (when it was written to the DB, so it effectively means import time).
- Long connections are saved as several sessions (split on packet count and timeouts). Use Connections or SPI View to see the whole conversation.
- TLS 1.3 encrypts the server certificate, so `cert.*` is empty. Use SNI (`host`), `tls.ja3s` and `tls.ja4` instead.

## FIELDS Session-wide fields

| FIELD | HOLDS / EXAMPLE |
|---|---|
| `ip`  `ip.src`  `ip.dst` | all IP fields / source / destination |
| `port`  `port.src`  `port.dst` | ports |
| `ip.protocol` | `tcp udp icmp sctp` |
| `protocols` | app protocols seen: `protocols == smb` |
| `host` | every hostname field (DNS, HTTP Host, SNI, SMB, ...) |
| `user` | every username field (HTTP, SMB, Kerberos, LDAP, mail, ...) |
| `tags` | tags from import (`-t`) or added in the UI |
| `file` | source pcap path: `file == *host7*` |
| `node` | capture node that indexed it |
| `country.src`  `country.dst` | ISO-2 country (needs Geo files) |
| `asn.src`  `asn.dst`  `rir.dst` | AS number+name, registry: `asn.dst == *Amazon*` |
| `mac.src`  `mac.dst`  `oui.src` | MAC addresses, vendor |
| `bytes`  `bytes.src`  `bytes.dst` | bytes on the wire, per direction |
| `databytes(.src/.dst)` | payload bytes only |
| `packets(.src/.dst)` | packet counts |
| `session.length` | duration in ms: `> 3600000` = over 1 h |
| `starttime`  `stoptime` | first / last packet |
| `communityId` | Community ID flow hash, the same as Zeek/Suricata `community_id` |
| `huntId`  `huntName` | sessions matched by a packet Hunt |

## FIELDS TCP flags & first payload bytes

| EXPRESSION | MEANING |
|---|---|
| `tcpflags.syn / .syn-ack / .ack / .psh / .fin / .rst / .urg` | count of packets carrying that flag |
| `tcpflags.syn > 0 && tcpflags.syn-ack == 0` | SYN never answered (scan, dead host) |
| `tcpflags.rst > 0 && databytes == 0` | refused / reset, no data |
| `packets.dst == 0` | no reply at all |
| `payload8.src.hex` `payload8.dst.hex` | first 8 payload bytes per side, in hex |
| `payload8.src.hex == 16030*` | TLS ClientHello, on any port |
| `payload8.src.hex == 5353482d*` | "SSH-" banner, on any port |
| `payload8.dst.hex == 4d5a*` | "MZ": raw PE pushed over plain TCP |

## FIELDS DNS

| FIELD | HOLDS / EXAMPLE |
|---|---|
| `host.dns` | queried name: `host.dns == *.duckdns.org` |
| `ip.dns` | IPs returned in answers |
| `dns.query.type` | `A AAAA TXT MX NULL ANY PTR SRV` |
| `dns.status` | `NOERROR NXDOMAIN SERVFAIL REFUSED` |
| `dns.query.class` | `IN`; `CH` = version.bind probing |
| `dns.opcode` | `QUERY`, `UPDATE` (dynamic DNS changes) |
| `host.dns.nameserver` `ip.dns.nameserver` | NS answers |
| `host.dns.mailserver` | MX answers |

## FIELDS HTTP

| FIELD | HOLDS / EXAMPLE |
|---|---|
| `host.http` | Host header |
| `http.uri` | host + path + query, so wrap patterns: `*login*` |
| `http.uri.path` | path only |
| `http.uri.key`  `http.uri.value` | query-string names / values: `http.uri.key == cmd` |
| `http.method`  `http.statuscode` | `POST`, `CONNECT`; `200`, `404` |
| `http.user-agent` | User-Agent string |
| `http.request-referer / -authorization / -content-type / -origin` | request headers (default config.ini set) |
| `http.response-server / -location / -content-type` | response headers; `-location` = redirects |
| `http.hasheader.src` `http.hasheader.dst` | header *names* present |
| `http.authtype`  `http.user` | `basic digest ntlm negotiate bearer`; user |
| `http.cookie.key` `http.cookie.value` | cookies |
| `http.bodymagic` | libmagic type of bodies: `*dosexec*  *zip*` |
| `http.md5`  `http.sha256` | body hashes (sha256 needs `supportSha256=true`) |
| `http.reqbody` | start of the request body (POST data) |
| `http.version`  `ip.xff` | HTTP version; X-Forwarded-For IPs |

## FIELDS TLS & certificates

| FIELD | HOLDS / EXAMPLE |
|---|---|
| `host` | SNI lands in the host fields |
| `tls.version` | `SSLv3 TLSv1 TLSv1.2 TLSv1.3` |
| `tls.cipher` | negotiated cipher suite |
| `tls.ja3`  `tls.ja3s`  `tls.ja4` | client / server / JA4 fingerprints |
| `cert.subject.cn` `cert.subject.on` | subject CN / organisation |
| `cert.issuer.cn`  `cert.issuer.on` | issuer CN / organisation |

## FIELDS TLS & certificates (cont.)

| FIELD | HOLDS / EXAMPLE |
|---|---|
| `cert.alt` | Subject Alternative Names |
| `cert.serial` `cert.hash` | serial / SHA-1 fingerprint |
| `cert.notbefore` `cert.notafter` | validity window |
| `cert.validfor` `cert.remainingDays` | lifetime in days / days left when seen |
| `cert.publicAlgorithm` `cert.curve` | key type / curve |

## FIELDS SMB, Kerberos, LDAP

| FIELD | HOLDS / EXAMPLE |
|---|---|
| `smb.share` | `\\HOST\SHARE`: `smb.share == *ADMIN$` |
| `smb.fn` | file names opened / written |
| `smb.user` `smb.domain` `smb.host` | NTLMSSP identity of the client |
| `smb.os` `smb.ver` | client OS / version strings (SMB1) |
| `krb5.cname` `krb5.sname` `krb5.realm` | client principal, service (`krbtgt/`, `cifs/`, `host/`), realm |
| `ldap.authtype` `ldap.bindname` | `simple` = cleartext password; bind DN |

## FIELDS SSH, mail, DHCP, ICMP, proxies, DBs

| FIELD | HOLDS / EXAMPLE |
|---|---|
| `ssh.ver` | banners: `*OpenSSH*` `*libssh*` `*paramiko*` `*Go*` |
| `ssh.hassh` `ssh.hasshServer` `ssh.key` | HASSH client / server, host key |
| `email.src` `email.dst` `email.subject` | SMTP envelope + subject (cleartext only) |
| `email.fn` `email.md5` `email.content-type` | attachments |
| `email.x-mailer` `host.email` `ip.email` | mail client, hosts/IPs in headers |
| `dhcp.host` `dhcp.mac` `dhcp.oui` `dhcp.type` | hostname to MAC to IP mapping |
| `icmp.type` `icmp.code` | 8 echo req, 0 reply, 3 unreachable, 11 TTL |
| `socks.user` `host.socks` `ip.socks` `port.socks` | SOCKS proxy target |
| `mysql.user` `postgresql.user` `postgresql.db` `oracle.user` | database logins |
| `irc.nick` `irc.channel` | IRC (legacy botnets) |

Common `protocols` values: tcp udp icmp http http2 tls dns smb ssh smtp ftp imap pop3 krb5 ldap dhcp ntp quic socks irc mysql postgresql tds redis mongo bittorrent. SPI View › Protocols lists the ones present in your data.

## HUNT Hunting expressions: web & delivery

| EXPRESSION | LOOKS FOR |
|---|---|
| `ip.dst != [10.0.0.0/8,172.16.0.0/12,192.168.0.0/16]` | leaves RFC1918 (save as Shortcut `$internal`) |
| `http.bodymagic == *dosexec* \|\| http.uri == [*.exe,*.dll,*.ps1,*.hta,*.vbs,*.js,*.bat,*.scr,*.msi]` | executable / script downloads |
| `http.user-agent == [*PowerShell*,curl*,Wget*,python*,*Python-urllib*,Go-http-client*,*CryptoAPI*,*BITS*]` | scripted / LOLBin clients (certutil = CryptoAPI, bitsadmin = BITS) |
| `host.http == /[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+(:[0-9]+)?/` | HTTP to a bare IP |
| `http.method == POST` | then SPI View › host.http, rarest first |
| `http.uri.key == [cmd,exec,command,c,q]` | web-shell style parameters |
| `http.statuscode == [401,403]` | auth failures / brute force (SPI on ip.src) |
| `http.authtype == basic \|\| ldap.authtype == simple` | cleartext credentials |
| `port.dst == [21,23,110,143]` | FTP, Telnet, POP3, IMAP cleartext |
| `http.response-location == EXISTS! && http.statuscode == [301,302]` | redirect chains (phish / exploit kits) |

## HUNT Hunting expressions: C2, tunnels, exfil

| EXPRESSION | LOOKS FOR |
|---|---|
| `port.dst == 443 && protocols != tls` | non-TLS on 443 (custom C2, tunnels) |
| `protocols == tls && port.dst != [443,8443,993,995,465,636]` | TLS on odd ports |
| `protocols == http && port.dst != [80,8080,8000,3128]` | HTTP on odd ports |
| `protocols == ssh && port.dst != 22` | SSH on odd ports / reverse tunnels |
| `tls.version == [SSLv3,TLSv1]` | legacy TLS (old malware, old kit) |
| `cert.remainingDays < 0` | expired certificates in use |
| `cert.issuer.cn == /www\.[a-z0-9]+\.com/ && cert.subject.cn == /www\.[a-z0-9]+\.net/` | Tor-style random certificates (heuristic) |
| `tls.ja3 == <md5>` `tls.ja4 == <ja4>` | known-bad fingerprints from intel |
| `bytes.src > 50000000` | client uploaded > 50 MB (exfil) |
| `session.length > 3600000` | sessions over 1 h |
| `ip.protocol == icmp && databytes > 10000` | ICMP carrying data (tunnelling) |
| `tcpflags.syn > 0 && tcpflags.syn-ack == 0` | scanning: SPI View ip.src × port.dst |

**Beacons:** narrow to one `ip.src`, then use SPI Graph on `ip.dst` or `host`. Evenly spaced bars at a low, steady rate suggest a beacon. Confirm in Sessions sorted by start time.

## HUNT Hunting expressions: DNS & Windows lateral movement

| EXPRESSION | LOOKS FOR |
|---|---|
| `dns.status == NXDOMAIN` | DGA / dead C2: SPI View ip.src |
| `host.dns == /.{60,}/` | very long names (DNS tunnelling) |
| `dns.query.type == [TXT,NULL]` | TXT / NULL heavy talkers |
| `protocols == dns && port.dst != 53` | DNS on a non-standard port |
| `protocols == dns && ip.dst != [10.0.0.53]` | clients bypassing your resolvers (edit list) |
| `host.dns == [*.duckdns.org,*.no-ip.*,*.ddns.net,*.hopto.org,*.dynu.com]` | dynamic-DNS providers |
| `host.dns == *xn--*` | punycode / homoglyph domains |
| `smb.share == [*ADMIN$,*C$,*IPC$]` | admin-share use |
| `smb.fn == [*.exe,*.dll,*.ps1,*.bat,*PSEXESVC*]` | tool drops / PsExec |
| `port.dst == [3389,5985,5986,135,445] && ip.src == $internal` | RDP / WinRM / RPC / SMB host to host |
| `port.dst == 3389 && ip.src != $internal` | RDP from outside |
| `krb5.sname == *krbtgt*` | TGT requests, to scope Kerberos activity |

## VIEWER UI pages & workflow

| PAGE | USE IT FOR |
|---|---|
| `Sessions` | Session table. ▸ expands the SPI detail. Click any value and choose *and* / *or* / *and not* to add it to the query. Save layouts as Views. |
| `SPI View` | Top values and counts for every field over the current query: who, what and where at a glance. |
| `SPI Graph` | One field's top values over time. Shows beacon cadence and bursts. |
| `Connections` | Node graph. Set Src/Dst fields (e.g. `ip.src` to `ip.dst:port`). Shows lateral movement and fan-out. |
| `Hunt` | Search raw packet bytes (ASCII, hex or regex; src, dst or both) in the query's sessions. Matches get `huntId`. Slow, so narrow the query first. |
| `Files` | PCAP files loaded per node. |
| `History` | Every search run, with user and time. Useful for case notes. |
| `Stats` | Capture / OpenSearch health and indices. |
| `Settings` | Views, column sets, **Shortcuts** (`$name` value lists), theme. |

### SESSION DETAIL

- Packet view: natural / ascii / utf-8 / hex, line numbers, "Show images & files" (renders carved content), uncompress gzip.
- Download src bytes, dst bytes or the raw PCAP. "Open in CyberChef" uses a copy bundled with the viewer, so it works offline.

### ACTIONS MENU (SESSIONS)

- Export PCAP / CSV for open, visible or all matching sessions. Add or remove tags (e.g. `case42-evil`). Remove data.

## AUTOMATE URL parameters & API

| PARAM | MEANING |
|---|---|
| `expression` | the search expression (URL-encoded) |
| `date` | hours back: 1 6 24 48 72 168 720; **-1 = all time** |
| `startTime` `stopTime` | epoch seconds (overrides date) |
| `bounding` | `last first both either database` |
| `length` `view` | rows to return; saved View name |

```
# digest auth; -G + --data-urlencode avoids hand-encoding
U=admin:arkime; A=http://localhost:8005
# matching sessions as CSV
curl -s --digest -u $U -G "$A/api/sessions/csv" \
  --data-urlencode 'date=-1' \
  --data-urlencode 'expression=ip.src == 10.1.1.5 && port.dst ==
   443' -o s.csv
# PCAP of matching sessions
curl -s --digest -u $U -G "$A/api/sessions/pcap" \
  --data-urlencode 'date=-1' --data-urlencode 'expression=tags ==
   case42' -o case42.pcap
# unique values + counts (exp = field to count)
curl -s --digest -u $U -G "$A/api/unique" --data-urlencode 'date=-1'
   \
  --data-urlencode 'exp=host.dns' --data-urlencode 'counts=1' \
  --data-urlencode 'expression=dns.status == NXDOMAIN'
# JSON (pipe to jq)
curl -s --digest -u $U -G "$A/api/sessions" --data-urlencode
   'date=-1' \
  --data-urlencode 'length=500' --data-urlencode
   'expression=protocols == smb'
```

Older builds use /sessions.csv, /sessions.pcap and /unique.txt.

## CLI Offline PCAP import (dfd VM)

```
C=/opt/arkime/etc/config.ini
# one pcap: copy into Arkime's store, tag it
sudo /opt/arkime/bin/capture -c $C -r /cases/42/host7.pcap --copy -t
   case42 -t host7
# a folder, recursive, skip files already loaded
sudo /opt/arkime/bin/capture -c $C -R /cases/42/pcap --recursive --
   skip --copy -t case42
# if capture rejects a pcapng
editcap -F pcap in.pcapng out.pcap
```

| FLAG | MEANING |
|---|---|
| `-r FILE` | read a pcap (repeatable) |
| `-R DIR` `--recursive` | read every pcap in DIR / include subfolders |
| `--copy` | copy into Arkime's pcap dir. Without it the viewer reads the original path, which must stay mounted. |
| `--skip` | skip files already in the DB |
| `-t TAG` | tag every session (repeatable) |
| `-n NODE` | node name to record against |
| `--flush` | flush sessions between files |
| `--delete` | **deletes the source pcap after import**. Never use it on evidence. |

## ADMIN Health, users, reset (dfd VM)

```
# dfd VM: viewer http://localhost:8005 (admin / arkime), OpenSearch
   127.0.0.1:9200
sudo systemctl status arkimeviewer
sudo journalctl -u arkimeviewer -e
tail -f /opt/arkime/logs/viewer.log       # capture.log for imports
curl -s '127.0.0.1:9200/_cluster/health?pretty'
curl -s '127.0.0.1:9200/_cat/indices?v&s=index'
D=/opt/arkime/db/db.pl; ES=http://127.0.0.1:9200
$D $ES info                               # versions, counts, sizes
$D $ES wipe                               # drop ALL sessions;
   keeps users/settings
$D $ES init                               # DANGER: rebuild schema,
   users lost
sudo /opt/arkime/bin/arkime_add_user.sh <user> "<Full Name>" <pass>
   --admin
```

**Disk full → OpenSearch read-only:** free space, then `curl -XPUT 127.0.0.1:9200/_all/_settings -H 'Content-Type: application/json' -d '{"index.blocks.read_only_allow_delete":null}'`

**No results?** Check the time range (All) and Bounding, that the import finished (capture.log, Files page), and that OpenSearch health is not red.